复现说明报告

在 PiPER 标准版上复现 FACTR 2

从 10 分钟自由运动数据到接触感知策略：一台无力矩传感器的机械臂怎么把论文做出来

NEXT 神经外部力矩估计 • FIRST 力信息驱动重采样 • 自由空间 L1 目标 ≤ 0.05 N•m

论　　文 FACTR 2: Learning External Force Sensing for Commodity Robot Arms Improves Policy Learning

作　　者 S. Oh, J. J. Liu, T. Tao, P. Han, K. Shaw, S. Funabashi, R. Salakhutdinov, D. Pathak（共同第一作者）

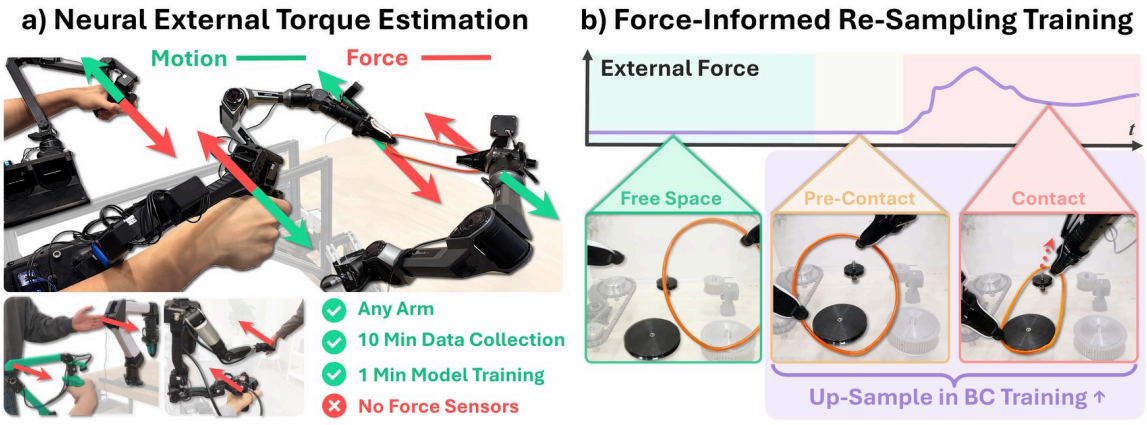
机　　构 Carnegie Mellon University · Waseda University

来　　源 arXiv:2606.12406v2 [cs.RO]　项目页 jasonjzliu.com/factr2

复现硬件 AgileX PiPER 标准版：6 DoF，1.5 kg 负载，626.75 mm 臂展，重复精度 ±0.1 mm，无关节力矩传感器，CAN 1 Mbps

报告定位 面向「动手复现」的施工图：能复现什么、不能复现什么、每一步怎么验收、坑在哪里

本报告回答三个问题：这台机器能复现到哪一层？每一步怎么验收？论文的数字为什么在你这里不会一模一样？



论文图 1：NEXT 用 10 分钟自由运动数据在无外力传感器的低成本机械臂上估计外部关节力矩；FIRST 再用该信号把示教切成自由运动 / 接触前 / 接触三段并重采样。

摘要

FACTR 2 的核心主张只有一句：没有力/力矩传感器的普通机械臂，也能"学会"感知外力，而且这个信号能显著提升接触富任务的策略学习效果。

它由两个模块组成。NEXT（Neural External Torque Estimation）用一个两层 LSTM，仅凭 10 分钟自由运动数据、1 分钟训练，就学会预测「无接触时本该出现的关节力矩」，再用实测电机力矩减去它，得到外部力矩估计。FIRST（Force-Informed Re-Sampling Training）拿这个估计把示教轨迹切成自由运动 / 接触前 / 接触三段，训练时只改采样分布、不改损失函数，把接触前和接触段样本的采样概率抬高。

对你这台 PiPER 标准版来说，结论是明确的：

- 论文本身就用 Piper 做实验平台（双臂 Piper 完成 5 个长时序任务；单臂 Piper 完成自由空间力矩误差评估与遥操作用户研究）。你的硬件与论文的低成本平台完全同款，不存在"机型对不上"的问题。
- 最硬的那条指标可以直接复现：论文在 Piper 自由空间上报告外部力矩 L1 误差 0.018 ± 0.012 N·m（FILIC 0.435、扰动观测器 0.248）。你的目标是同一张表。
- 接触段精度无法直接复现：Piper 没有力矩传感器，论文自己也只在 Franka 上做接触段定量评估，在 Piper 上改为「遥操作用户研究」这种主观评价。你若要定量，需要自建标定台（砝码 / 弹簧 / 外部力传感器）。
- 策略层（FIRST）需要重资产：论文每任务 250 条示教、双臂、4 路 RGB、20 轮 flow-matching 训练。单臂单任务的最小可行版本可以做，但结论只能作趋势验证，不能对标 84.9% 的任务进度。

一句话建议：先做 L1（自由空间 NEXT），这是投入产出比最高、且能真正验证"论文在你机器上成立"的一步；再决定是否投入 L4（策略训练）。

---

1. 这项研究解决什么问题

1.1 力觉很贵，而接触任务离不开它

插装、拧紧、擦拭、柔性物体操作——这些任务的成功与否取决于「接触上了没有、压多紧、该不该停」。但六维力/力矩传感器基本只出现在 Franka（约 3 万美元）、Flexiv、KUKA 这类平台上。Piper 整机约 2000 美元，给它加装末端力传感器，成本可能接近机械臂本身。

1.2 既有的「无力传感器」路线各有短板

- 解析法 + 系统辨识：需要准确的 M(q)、C(q,q)、g(q)，而残差里混着非线性摩擦、静摩擦、背隙、迟滞、温漂、力矩纹波、死区、饱和——这些是历史相关效应，瞬时状态观测不到。
- 自编码器异常检测：只能回答「有没有碰撞」，给不出方向和大小。
- 监督式接触估计：需要额外标注接触数据。

1.3 论文的赌注

把 τ_f（自由空间力矩）当成一个可以学习的函数，而不是一个必须解析建模的量。网络见过足够多的自由运动后，自然把摩擦、背隙、执行器动力学的「历史依赖」吸收进隐状态。于是：

```
τ_ext = τ_m - f_θ(x)
```

只要 f_θ 足够准，不需要任何力传感器。这是全文的地基。

---

2. 方法拆解

2.1 NEXT：把自由空间逆动力学学出来

论文的出发点是一个恒等式：

$$\tau_{ext} = \tau_m - \tau_f, \qquad \tau_f = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + \tau_{other}$$

其中 $\tau_m = K \cdot I_m$，I_m 是电机电流，K 是力矩常数。传统做法是解析算 τ_f，论文改为学：

$$\hat{\tau}_f = f_\theta(x_i), \qquad \hat{\tau}_{ext} = \tau_m - \hat{\tau}_f$$
$$x_i = [\, q_{i-H:i},\ \dot{q}_{i-H:i},\ \Delta q_{d,i-H:i} \,]$$
$$L(\theta) = (1/N) \sum \| f_\theta(x_i) - \tau_{m,i} \|^2$$

三个设计选择值得注意：

- 监督标签免费：自由运动时 $\tau_{ext} \equiv 0$，因此 τ_m 本身就是 τ_f 的标签。不需要力传感器、不需要负载标定、不需要人工辨识参数。
- 必须带历史：$\Delta q_d = q_d - q$（指令位置与实测位置之差）提供了「控制器有多努力」的线索，是纯刚体动力学里没有的量。附录消融显示 $(q, \dot{q}, \Delta q_d)$ 一致优于 $(q, \dot{q})$ 与单 q。
- 无状态滑窗：每条序列重置隐状态，用最近 H 步的滚动缓冲区推理，避免长序列隐状态漂移。

模型与训练超参（论文附录 A.1）：

项目	设定
循环骨干	2 层 LSTM，隐层 128
预测头	2 层 MLP，隐层 256
Dropout	0.1
损失	MSE
优化器	AdamW，lr 1e-3，weight decay 1e-6
梯度裁剪	1.0
早停	验证损失，patience 20，warmup 10，min Δ 1e-5
历史长度	H = 50（@100 Hz，即 0.5 s）
数据频率	100 Hz
训练成本	单张 RTX 3090 约 1 分钟
推理成本	单核 i9-10920X 1.76 ms/次（≈568 Hz），实际跑 100 Hz

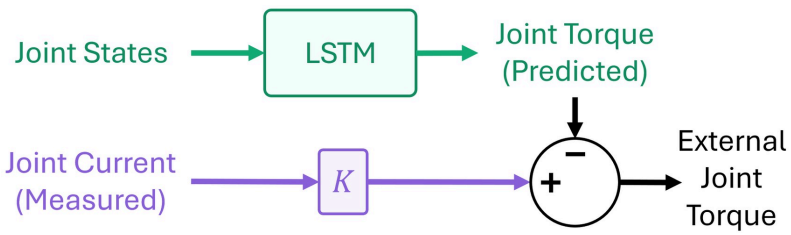


图 1　NEXT 的部署链路：电流 × 力矩常数得到实测力矩，LSTM 预测自由空间力矩，二者相减得到外部力矩。

2.2 FIRST：只改采样，不改损失

观察：接触富任务的失败点集中在接触附近——接触前要对准，接触中要控制力。而示教数据里，自由运动段往往占绝大多数，模型把容量浪费在「空挥」上。

做法分三步：

1. 接触打分：$f_t = \|\hat{\tau}_{ext,t}\|_1$，双阈值滞回得到二值指示 c_t（$f_t \geq T_{high}$ 置 1，$f_t \leq T_{low}$ 置 0，中间保持原值）。
2. 相位标注：接触起点 $T_{onset} = \{t : c_{t-1}=0, c_t=1\}$；接触点为 Contact，接触起点前 F 个控制周期（论文取 1 秒）为 Pre-Contact，其余为 Free Space。
3. 加权采样：给三个相位分配权重 w_F, w_{PC}, w_C，按 $p_t = w(s_t) / \Sigma\, w(s_j)$ 采样。损失函数一行不改。

策略侧接口：观测 $o_t = (M\text{ 路图像}, q_t, \hat{\tau}_{ext,t})$，输出动作块长度 30，用 flow matching 速度场目标训练；图像编码器 DINOv3-Base，动作空间是绝对关节角 + 夹爪开度。

2.3 论文的主要结果

外部力矩估计（Franka，L1 误差，N·m）

方法	接触段	自由空间
专用力矩传感器（真值）	0.000	0.449 ± 0.208
FILIC	4.395 ± 1.531	2.460 ± 1.358
扰动观测器（DO）	1.471 ± 0.761	2.429 ± 1.298
NEXT	0.547 ± 0.348	0.414 ± 0.278

Piper 自由空间（无接触，真值为 0）

方法	L1 误差（N•m）
FILIC	0.435 ± 0.225
扰动观测器	0.248 ± 0.132
NEXT	0.018 ± 0.012

Piper 上的误差比 Franka 低一个数量级，原因不神秘：Piper 更轻、更小、传动更简单，力矩量级更小，同一绝对误差就显得更小。这一点在复现时很重要——你追的是 0.018 这个数，不是 0.414。

五个长时序接触富任务（任务进度）

任务	Base	+Torque	FACTR	TA-VLA	FIRST
LEGO 装配	≈39%	≈65%	≈57%	≈63%	91.3%
NIST 皮带装配	≈15%	≈43%	≈36%	≈41%	76.7%
NIST 插装	≈79%	≈87%	≈86%	≈88%	93.3%
工具归位	≈46%	≈53%	≈62%	≈69%	80.5%
拧瓶盖	≈43%	≈56%	≈75%	≈78%	82.5%

平均	≈44%	≈61%	≈63%	≈68%	84.9%

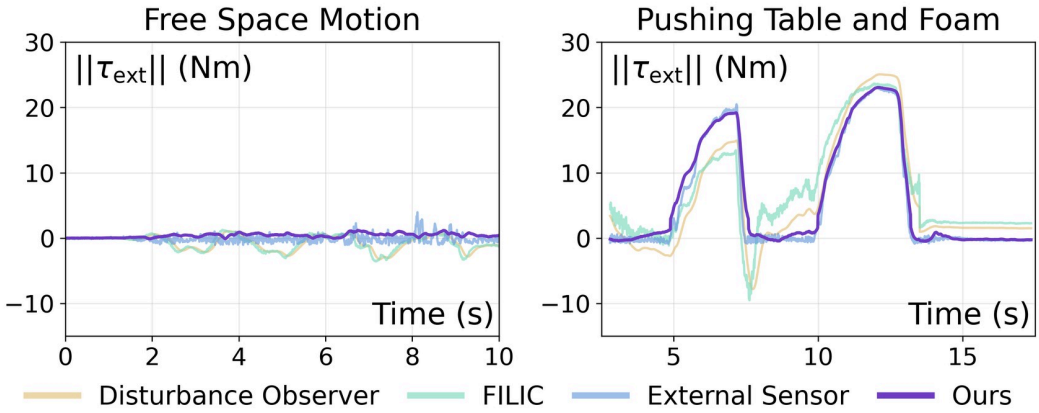


图 2　接触段的力矩估计曲线：NEXT 更平滑、更贴合零，FILIC 噪声大，DO 滞后。

3. 论文里的 Piper 究竟承担了什么

这一节决定了「你能复现到哪一步」。把论文里所有涉及 Piper 的实验拎出来看：

实验	平台	评价方式	是否有真值	你能复现吗
自由空间外力矩误差（表 5）	单臂 Piper	L1 误差，真值恒为 0	是（0 是已知的）	能，且是最优先
接触段外力矩误差（表 4）	Franka	与专用传感器对比	是	不能（Piper 无传感器）
力反馈遥操作用户研究（图 9）	Piper + 主从	1–5 分易用性 + 施加关节力矩	主观	能（需第二台或上级主臂）
五个长时序任务（表 2、图 6）	双臂 Piper + LEAP Hand / 夹爪	任务进度，每策略 20 次 rollout	人工评分	部分（单臂单任务可行）
相位消融、上采样倍数（附录 D）	双臂 Piper	任务进度	人工评分	部分
数据集覆盖度（图 8）	Piper	关节角、角速度、Δq、τ_f 分布	—	能，且是训练前的必要检查

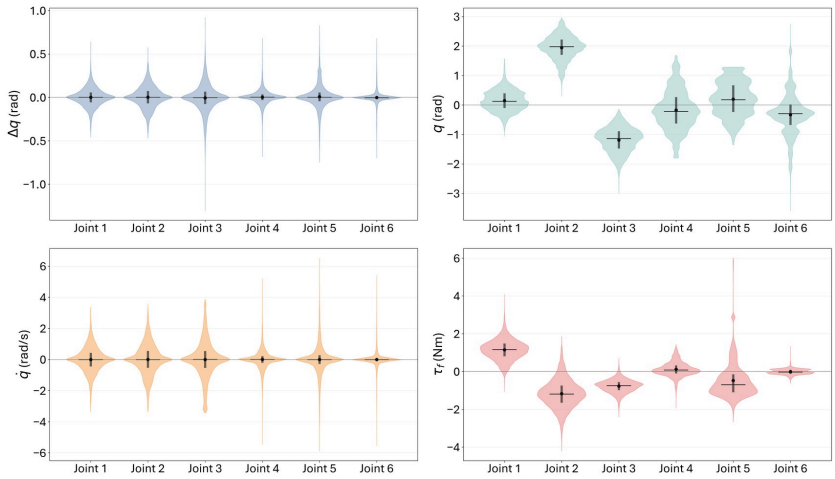


图 3　Piper 上的 10 分钟自由运动数据集覆盖情况：关节角、跟踪误差 Δq、角速度、自由空间力矩在六个关节上的分布。复现时应先把自己的数据集画成同样的四联图。

三条对复现有直接影响的结论：

1. Piper 的价值在于「无传感器也能估力矩」，论文没有、也无法在 Piper 上给出接触段的定量精度。所以任何声称「在 Piper 上复现出 0.547 N·m 接触误差」的说法都是错的——那是 Franka 的数字。
2. 策略实验用的是双臂。你要做完整五任务，需要两台 Piper + 两种末端（夹爪与 LEAP Hand）。单台只能做单臂改写版。
3. 遥操作用户研究需要主从两台（或一台 GELLO 类主臂）。`0x470` 的主从联动指令让这件事在原厂生态里变得简单。

---

4. 你的硬件：PiPER 标准版能力画像

完整参数与使用说明见同仓库 `piper_docs/PiPER_标准版_官方参数与使用说明.md`，这里只列与复现强相关的部分。

维度	参数	对复现的含义
自由度	6	与论文一致（论文动作空间为 6 关节 + 夹爪）
负载	1.5 kg（含末端）	装 0.5 kg 夹爪后仅余 ~1 kg，LEAP Hand 类灵巧手需核算
臂展	626.75 mm	桌面级任务够用；NIST 板类任务需合理布局
重复定位精度	±0.1 mm	插装类任务够用，不是瓶颈
关节范围	J1 ±154°，J2 0~195°，J3 -175~0°，J4 ±102°，J5 ±75°，J6 ±120°	非对称，归一化与采样需按实际限位；官方资料改过三版，以 `0x473` 回读为准
最大速度	180~225 °/s	快速接触会被限速，接触瞬态可能被平滑
力矩感知	无关节力矩传感器	只能靠电流 `I_m` × 力矩常数 `K`
电流反馈	`0x251~0x256`，20 ms（50 Hz），0.001 A	论文是 100 Hz，需对齐策略
关节角反馈	`0x2A5~0x2A7`，0.001°；SDK 日志含 Hz	实际 100~200 Hz，需实测
通信	CAN 2.0B，1 Mbps，单总线	带宽充裕；双臂可用 ID 偏移共用一条总线
碰撞保护	`0x47A` 可设等级	触发时会改变力矩行为，采集时需固定等级
控制模式	位置-速度模式 / MIT 模式（`0x15A~0x15F`，含 T_ref）	MIT 模式可做力矩前馈，是力反馈遥操作的关键

4.1 三个硬约束

约束一：力矩只能从电流来，而电流只有 50 Hz。 论文的信号链是 `τ_m = K·I_m`，Piper 的 `I_m` 在高

速反馈帧里，20 ms 一帧。这意味着你的数据在电流维度上是 50 Hz。两种处理：整套管线降到 50 Hz（推荐，`H=25` 对应 1 秒历史），或对电流做零阶保持/线性插值升到 100 Hz（需在论文/记录中写明）。

约束二：没有真值，就没有接触段的定量评估。 论文在 Franka 上能算 `|τ^ext − τ_ext^GT|`，在 Piper 上只能算自由空间（真值为 0）。你若想在 Piper 上给出接触精度，必须自建参考：已知质量的砝码悬挂、已知刚度的弹簧压缩、或外购一维/六维力传感器做一次性标定。没有参考就不要报接触误差数字。

约束三：负载预算是全链条的。 1.5 kg 包含夹爪、相机支架、线缆。论文的双臂任务里还挂了 LEAP Hand V2。若你打算复刻 LEGO 装配，先称重。

约束四：想加外部力传感器，别打末端 CAN 的主意。 Piper 末端 XT30 2+2 只开放 24 V / 2 A 供电与松灵自有设备的 CAN，官方明确「严禁私自接入其他 CAN 设备」。要用六维力传感器做接触段真值，必须走独立总线/独立 USB 接口，并在负载预算里算上它的重量。

---

5. 逐项对照：论文设定 vs 你的机器

这是本报告的核心表格。左列是论文的做法，中列是你手上的实际条件，右列是差异与应对。

#	论文设定	Piper 标准版实际	差异与应对
1	Franka Panda 提供关节力矩传感器与出厂外部力矩估计，作为接触段真值	无任何力矩传感器	不可复现接触段定量指标；改用「自由空间真值为 0」+ 自建标定台
2	外力矩 `τ_m = K·I_m`，`K` 由厂家提供或电流-力矩曲线得到	电流可得（0.001 A），SDK 另有 `effort` 字段（固定系数换算）	`K` 需自行标定；`effort` 可作初值，但不要盲信，见 7.3 节
3	数据 100 Hz	电流 50 Hz，状态 100~200 Hz	管线跑 50 Hz（`H=25`）或插值升频；两种都要在记录里写明
4	10 分钟自由运动，覆盖单关节全行程 → 多关节类笛卡尔 → 快慢速重复	可用 API 自动跑轨迹；拖动示教单条限 3 分钟	优先用 API 脚本自动采集（可重复、可覆盖、无人工抖动）；示教仅作补充
5	双臂 Piper，每任务 250 条示教，4 路 RGB（2 腕 + 2 顶视）	单台标准版（按你的描述）	五个任务全部需要双臂；单台只能做单臂改写任务，或再购一台 + `0x470` 单总线双臂
6	力反馈遥操作：`τ_feedback = K_{f,p}·τ_ext` 施加到主臂	需第二台 Piper 作主臂（`0xFA` 示教输入 / `0xFC` 运动输出）	双臂主从联动；或先做「无力反馈的位置-位置」对照，验证管线
7	策略：flow matching DiT，DINOv3-Base，动作块 30，20 epoch	需 GPU（论文用 3090 级）	单任务最小可行：50~80 条示教 + ACT 或小 DiT
8	关节空间对称假设（Franka 各关节近 ±180°）	J2 0~195°、J3 -175~0°	归一化按各关节实际限位；采样时按限位裁剪，避免生成不可达动作

9	接触检测阈值 `T_high` / `T_low` 需按机型标定	论文未给具体数值	用分位数法（如 97/90 分位）+ 人工接触标注校验，见 7.5 节
10	碰撞保护不构成干扰（Franka 为力矩级保护）	Piper 有 `0x47A` 碰撞防护等级，触发时改变力矩行为	采集期间固定等级并记录；若频繁触发，说明轨迹过激或阈值过保守
11	负载充裕（Franka 3 kg）	1.5 kg 含末端	LEGO 任务换轻量手；必要时降低加速度与速度百分比
12	控制器与动力学模型由厂家 URDF 提供（FILIC 基线用 MuJoCo）	官方 URDF 有新旧两版（固件 S-V1.6-3 起 J2/J3 偏移 2°）	用与固件匹配的 URDF，否则 FILIC/DO 基线会系统性偏差

> 一句话总结：硬件层面你「完全在论文的靶心内」——Piper 就是论文的低成本平台；差距不在机械臂，而在真值、第二台臂、以及策略训练的数据与算力。

---

6. 可复现性分级

把论文拆成五层，按投入从小到大排列。建议逐层验收，不要跳级。

层级	目标	依赖	周期	验收指标
L0 信号链	打通 CAN → 时间戳 → 落盘，能稳定读 `q, q̇, I_m, τ_m`	一台 Piper + USB-CAN + Ubuntu	1~2 天	10 分钟连续采集零丢帧；时间戳单调；50 Hz 实际频率 ≥ 49 Hz
L1 NEXT（自由空间）	训练 LSTM，复现表 5 量级	L0 + 10 min 数据 + 单卡 GPU	2~4 天	留出集 L1 误差 ≤ 0.05 N·m（论文 0.018）；静止时 `τ̂ext ≈ 0`；不与指令相关
L2 接触检测 + FIRST 标注	阈值标定、相位切分、加权采样器	L1	1~2 天	20 次人工接触，检出率 ≥ 90%，误报 ≤ 1 次/分钟；相位分布合理（接触前 ≈1 s）
L3 力反馈遥操作	主从联动 + `τ_feedback`	L1 + 第二台（或主臂）	3~5 天	与「无反馈」「位置-位置」对比，主观评分更高、施加关节力矩更低
L4 策略学习	单任务（或五任务）FIRST vs Base	L2 + 双臂 + 相机 + GPU + 250 条示教/任务	数周	单任务任务进度较 Base 提升 ≥ 10 个百分点（论文平均 +41 pt）

投入产出判断：L0+L1 用一台机器、一张消费级显卡、不到一周，就能得到「论文结论在我这台机器上成立」的硬证据。L4 是论文的主要卖点，但也是成本最高、最容易受数据质量影响的一层。

---

7. 复现路线图

7.1 Phase 0：环境与信号链

目标：把「机械臂 → CAN → Python → 磁盘」这条链路做成可重复、可校验的。

步骤

1. 硬件：Piper 标准版 + 官方 USB-CAN + 24 V/10 A 电源；底座固定在刚性台面，臂展范围内清空。
2. 系统：Ubuntu 20.04/22.04（官方 SDK 测试平台），`can-utils`、`python-can`、`piper_sdk`。
3. 起 CAN：`piper-can activate can0 1000000`，`ifconfig can0` 确认。
4. 读固件：`GetPiperFirmwareVersion()`；≥1.8-8 用 `C_PiperInterface_V3`；确认 URDF 新旧版本（S-V1.6-3 分界）。
5. 切模式：示教灯熄灭 → 上位机待机 → CAN 指令控制模式；回零位。
6. 使能：`EnableArm(7)`。
7. 写采集脚本（附录 B.1）：单线程读 CAN、独立线程写盘、统一时间戳（`time.perf_counter()` + 单调时钟）。

必须做的三项验收

- 首帧检查：丢弃第一帧（默认值全 0），并断言角度不全为 0。
- 频率检查：统计 `0x251` 实际到达间隔，均值 20 ± 2 ms。
- 零位检查：静止 30 s，记录 τ_m 的均值与标准差——这是你后面判断「残差是否贴零」的基线。

> 坑：`GetArmHighSpdInfoMsgs()` 的电流字段是 0.001 A 的整数，忘了除 1000 会让 K 标定差三个数量级。SDK 的 `GetArmLowSpdInfoMsgs()` 里的「母线电流」在 1.5 kg 机型上恒为 0（官方协议注明无采样），别拿它当电流源。

7.2 Phase 1：自由运动数据采集（10 分钟）

论文的采集规程很朴素，照做即可：

1. 单关节全行程：J1~J6 各自从限位一端到另一端，慢速、匀速各一遍。
2. 多关节类笛卡尔：末端做直线、圆弧、往复，让多个关节联动。
3. 快慢速重复：同一轨迹用 20%、50%、100% 速度各跑一遍，覆盖速度相关动力学（摩擦、阻尼）。

推荐用 API 自动跑（而非拖动示教）：可重复、覆盖均匀、无人工抖动，且能精确记录 q_d（指令位置）。示教数据可另存一份作对照。

采集参数建议

项	建议值	理由
采样率	50 Hz（对齐电流）	避免插值伪影
时长	10~15 分钟	论文 10 分钟够用，多留冗余
速度百分比	20 / 50 / 100 各一轮	覆盖速度相关项
加速度	默认或略低	过高会触发碰撞保护
碰撞保护等级	固定并记录	避免力矩行为漂移
环境温度	冷机启动后先空跑 5 分钟预热	温漂会影响 τ_f

目录结构建议（与论文的 $D_{free} = \{(x_i, \tau_{m,i})\}$ 对应）

```
data/piper_free/
├── raw/2026-10-19_run01/          # 原始 CAN 帧 + 解析后 npz
│   ├── can_frames.parquet
│   └── meta.json                  # 固件版本、URDF 版本、K、碰撞等级、室温
├── processed/free_50hz.npz        # q, qd, q_cmd, I, tau_m, t
└── figures/coverage.png           # 与论文图 8 同款的四联图
```

采集后必看：把自己数据的 q、Δq、$\dot{q}$、τ_f 六个关节分布画成论文图 8 的样子。如果某个关节几乎没被激励（比如 J6 一直被夹爪任务占住），τ_f 在这一维上学不好，后面接触检测会漏。

7.3 Phase 2：力矩常数 K 标定

论文的局限里第一条就是「绝对力矩尺度依赖 K」。Piper 不给你 K，所以必须自己测。两个互补方法：

方法 A：静态重力法（推荐先做）

1. 卸下末端或装已知质量 m 的配重，测出质心到各关节轴的距离；
2. 让机械臂静止在若干姿态，记录稳态电流 I_{ss}；
3. 对每个关节，用静力矩平衡求解：

$$K_j \cdot I_{ss,j} = \Sigma_i \ m_i \cdot g \cdot r_{i,j}(\theta)$$

4. 多姿态最小二乘拟合 K_j（每关节一个标量）。注意扣除关节自身的重力项——所以至少要做「空载 + 加载」两组，差分消掉本体项：

$$K_j = \Delta\tau_j \ / \ \Delta I_j$$

方法 B：斜坡力矩法

用 MIT 模式（0x15A~0x15F）给某关节施加已知 T_{ref} 阶跃，读稳态电流，得到 $K_j = T_{ref} / I_{ss}$。注意：MIT 模式官方明确警告误用会损坏机械臂，Kp/Kd 需从小值开始，且必须在有人看护、行程中部、无障碍条件下进行。

验收：K_j 在同一关节不同姿态下的离散度 < 5%；六个关节的 K 量级应相近（同系列电机）。

7.4 Phase 3：训练 NEXT 并验收

数据切分：按「轨迹段」切，不要按帧随机切——否则相邻帧泄漏，验证误差假性偏低。建议 8:1:1（训练/验证/测试），且测试集包含「训练时未出现过的速度档」。

训练配置（复刻论文表 3）

项	值
输入	$[q, \dot{q}, \Delta q_d]$，H = 25（@50 Hz，等价论文 1 秒）
归一化	按训练集均值/方差，逐关节
模型	2×LSTM(128) + 2×MLP(256)，dropout 0.1
损失	MSE
优化	AdamW，lr 1e-3，wd 1e-6，clip 1.0

早停	patience 20，warmup 10
batch	256（按显存调）
目标	留出集 $\|\hat{\tau}_f - \tau_m\|_1 \le 0.05$ N·m

四道验收关

1. 绝对精度：留出集 L1 ≤ 0.05 N·m（论文 0.018，你的装置与标定不同，允许 2~3 倍）。
2. 贴零性：机械臂静止、无接触时，$\hat{\tau}_{ext}$ 的均值应 < 0.03 N·m，且不随姿态系统性漂移。
3. 无泄漏：把 $\dot{q}$ 置零再推理，误差应显著变差——若几乎不变，说明模型没用到速度信息，可能过拟合到位置。
4. 历史必要性：把 H 改成 1 重训，误差应明显上升（论文消融里 MLP/短历史更差）。若不变，说明你的数据多样性不足。

一个容易忽略的点：论文用「无状态滑窗、每条序列重置隐状态」。部署时如果按轨迹连续喂而不重置，隐状态会累积漂移，表现为「越跑越偏」。实现时务必按窗口重置。

7.5 Phase 4：接触检测与阈值标定

步骤

1. 用 L1 模型对一段包含真实接触的数据（手指按压、砝码放置、插装）推理，得到 $f_t = \|\hat{\tau}_{ext,t}\|_1$。
2. 画 f_t 的时间曲线，人工标出真实接触区间（10~20 次）。
3. 用分位数初始化阈值：$T_{high} = P97(f_t)$、$T_{low} = P90(f_t)$（自由段为主的数据上）。
4. 网格搜索 (T_{high}, T_{low})，最大化 F1；滞回带宽 $T_{high} - T_{low}$ 通常取 T_{high} 的 20%~40%。
5. 固定阈值并写入配置文件，连同标定数据与 F1 一起存档。

验收：检出率 ≥ 90%，误报 ≤ 1 次/分钟，接触起点延迟 < 100 ms（$F = 1$ s 的接触前窗口才有意义）。

> 注意：Piper 的碰撞保护（0x47A）在某些等级下会在检测到外力时主动卸力，这会让 τ_m 出现台阶——如果你的 f_t 在接触瞬间出现「尖峰后塌陷」，先检查保护等级，而不是怀疑模型。

7.6 Phase 5：FIRST 采样器

只改采样，不改损失。核心实现：

```
# 权重：自由 : 接触前 : 接触
W = {0: 1.0, 1: 5.0, 2: 1.0}       # 论文最优 1:5:1
# 持续接触型任务（如拧瓶盖）改 1:3:3

w = np.array([W[s] for s in phase_labels])          # 每条样本的权重
p = w / w.sum()
idx = np.random.choice(len(dataset), size=batch, p=p)   # 或 WeightedRandomSampler
```

论文的三个经验结论，直接照抄能省很多试错：

- 上采样倍数 $m \approx 5$ 最佳，更大饱和或退化；
- 只上采样接触前（PC）通常最好（5 任务里 4 个），只上采样接触（C）几乎无效；
- 例外是拧瓶盖这类持续接触任务，PC + C 更好（1:3:3）。

7.7 Phase 6：策略训练与评估（可选）

最小可行方案（单臂单任务）

项	论文	你的最小版
任务	5 个双臂长时序任务	1 个单臂接触富任务（如插销、拧盖）
示教	250 条/任务	50~80 条
相机	4 路（2 腕 + 2 顶视）	2 路（1 腕 + 1 顶视）
策略	flow matching DiT + DINOv3-Base	ACT（更省算力）或小 DiT
训练	20 epoch	按验证损失早停
评估	20 次 rollout，任务进度	20 次 rollout，成功/失败 + 分段进度

三个对照必须都跑：Base（无 τ）→ Base + τ（有 τ 但不重采样）→ FIRST（有 τ + 重采样）。缺了中间那组，就无法区分收益来自「力矩输入」还是「重采样」。

评估注意：论文的「任务进度」是人工分段打分（拧瓶盖还按过拧次数扣分），跨任务不可比。你自己评时先写死评分规则，最好双人独立打分取均值。

---

8. 风险清单

风险	表现	规避
`K` 不准	`τ̂_ext` 整体缩放偏移，自由空间不贴零	双方法交叉验证；先静态法再斜坡法
自由数据混入轻微接触	标签被污染，模型学会「有接触也不输出」	采集时保持末端悬空、远离台面；采集后检查 `τ_m` 是否有异常尖峰
电流 50 Hz 插值伪影	`τ̂_ext` 呈阶梯状，接触检测抖动	管线直接 50 Hz，不插值
温漂	长时间运行后 `τ_f` 漂移，误差缓慢上升	采集前预热 5 分钟；长实验中途重采一段校准
碰撞保护干预	接触瞬间力矩塌陷	固定 `0x47A` 等级并记录；必要时降一档
固件/URDF 不匹配	FILIC、DO 基线系统性偏差；DH 差 2°	记录固件版本；`dh_is_offset` 与 URDF 严格对应
双臂 ID 冲突	只有一台响应	`0x470` 设偏移（0x10/0x20），或两个 USB-CAN
使能状态残留	上次进程退出未失能，误发指令即动作	脚本 `try/finally` 里 `DisableArm(7)`
示教 3 分钟限制	长轨迹被截断	用 API 采集，或分段拼接
数据泄漏	验证误差虚低	按轨迹段切分，不按帧随机切

---

9. 成本与时间预算

项	已有	需新增	说明
Piper 标准版	已有 1 台	—	论文同款

USB-CAN 模块	已有（随机附）	—	双臂需 2 个，或单总线 ID 偏移
夹爪	视配置	若做抓取任务需 1 个	0.5 kg，占负载 1/3
第二台 Piper（L3/L4）	待购	1 台	主从遥操作与双臂任务必需
工控机 / 带 GPU 主机	视情况	1 台（3090 级）	L1 训练 1 分钟；L4 需数小时~数天
相机	视情况	1~4 个 RGB	论文 4 路
标定台（砝码/弹簧/力传感器）	待购	低成本可自建	仅在你需要接触段定量时
L0+L1 周期	—	约 1 周	单人，含调试
L2~L3 周期	—	约 1~2 周	依赖第二台臂
L4 周期	—	数周	数据采集占大头

---

10. 预期偏差：你需要提前接受的四件事

1. 数值不会等于 0.018 N·m。 这个数字绑定论文的装置、标定与数据分布。你的合理目标是同一量级、且显著优于 FILIC/DO 基线。若你的 FILIC 是 0.4、NEXT 是 0.05，趋势就复现了。
2. 接触段你拿不到真值。 除非自建参考，否则不要报接触误差；改用「检测一致性」「遥操作主观评分」「任务成功率」这类可观测指标。
3. 策略层的提升幅度会小于论文。 论文的 +41 个百分点（Base 44% → FIRST 85%）建立在 250 条高质量双臂示教 + 4 路视角之上。单臂 50 条示教的场景，能复现出「FIRST > Base + Torque > Base」的排序就已经是成功。
4. Piper 的 J2/J3 非对称会给策略带来额外难度。 论文的绝对关节角动作空间在对称机型上更"平"，Piper 上需要对不可达区域做裁剪，否则模型会学出撞限位的动作。

附录 A　符号与公式速查

符号	含义	单位
$q,\ \dot{q},\ \ddot{q}$	关节位置、速度、加速度	rad, rad/s, rad/s²
q_d	指令关节位置	rad
$\Delta q_d = q_d - q$	跟踪误差（控制器努力线索）	rad
I_m	电机电流	A
K	力矩常数	N·m/A
$\tau_m = K \cdot I_m$	实测电机力矩	N·m
τ_f	自由空间力矩（待学）	N·m
τ_{ext}	外部力矩（接触产生）	N·m
H	历史窗口长度	步
$f_t = \|\hat{\tau}_{ext,t}\|_1$	接触打分	N·m

T_{high}, T_{low}	滞回双阈值	N·m
F	接触前窗口（控制周期数）	步（论文 1 s）
w_F, w_{PC}, w_C	三相位采样权重	—

编号	公式
(1)	$\tau_{ext} = \tau_m - \tau_f,\ \tau_f = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + \tau_{other}$
(2)	$\hat{\tau}_f = f_\theta(x),\ \hat{\tau}_{ext} = \tau_m - \hat{\tau}_f$
(3)	$x_i = [q_{i-H:i},\ \dot{q}_{i-H:i},\ \Delta q_{d,i-H:i}]$
(4)	$L(\theta) = (1/N)\ \Sigma\ \|f_\theta(x_i) - \tau_{m,i}\|^2$
(5)	$L_{FM}(\theta) = E\|v_\theta(x_\alpha, \alpha, o_t) - (x_1 - x_0)\|^2$
(6)	$T_{onset} = \{t : c_{t-1}=0, c_t=1\}$
(7)	$s_t \in \{C, PC, FS\}$，PC 为 $t_{onset} - F \le t < t_{onset}$
(8)	$\tau_{feedback} = K_{f,p} \cdot \tau_{ext}$

附录 B　代码骨架

B.1 数据采集（50 Hz，电流与状态对齐）

```
"""piper_record.py — 采 10 分钟自由运动数据，落盘为 npz。
注意：先 EnableArm(7)，采完 DisableArm(7)；首帧丢弃。"""
import time, json, numpy as np
from piper_sdk import C_PiperInterface        # 固件 >=1.8-8 用 C_PiperInterface_V3

FREQ = 50.0
DT = 1.0 / FREQ

def main(out="free_50hz.npz", duration=600.0):
    piper = C_PiperInterface("can0", judge_flag=False)
    piper.ConnectPort()
    time.sleep(0.05)
    fw = piper.GetPiperFirmwareVersion()
    piper.EnableArm(7)
    time.sleep(0.2)

    buf = {k: [] for k in ("t", "q", "qd", "q_cmd", "I", "tau")}
    t0 = time.perf_counter()
    try:
        _ = piper.GetArmJointMsgs()          # 丢弃首帧（默认值全 0）
        next_t = t0
        while time.perf_counter() - t0 < duration:
            now = time.perf_counter()
            if now < next_t:
                time.sleep(min(next_t - now, DT / 4))
                continue
            next_t += DT

            js = piper.GetArmJointMsgs()
            hs = piper.GetArmHighSpdInfoMsgs()
            # SDK 单位为 0.001 度 / 0.001 A / 0.001 rad/s，务必换算
            q = np.array(js.joint_state.joint_1 ... joint_6, dtype=float) * 1e-3
```

```
            I = np.array([getattr(hs, f"motor_{i+1}").current for i in range(6)]) * 1e-3
            w = np.array([getattr(hs, f"motor_{i+1}").motor_speed for i in range(6)]) * 1e-3
            buf["t"].append(now - t0)
            buf["q"].append(q)
            buf["qd"].append(w)
            buf["q_cmd"].append(q)      # 位置-速度模式下用指令队列里最近的目标替换
            buf["I"].append(I)
            buf["tau"].append(I)        # 先用 I 存，离线乘 K
    finally:
        piper.DisableArm(7)
        piper.DisconnectPort()

    np.savez_compressed(out, **{k: np.asarray(v) for k, v in buf.items()})
    print("saved", out, "freq=", len(buf["t"]) / duration)

if __name__ == "__main__":
    main()
```

> 说明：`q_cmd` 必须来自你自己的指令队列（发出去的目标角），Piper 的反馈帧里没有「指令位置」字段。如果只用拖动示教，就没有 `q_cmd`，可用 `q` 的一阶保持近似，或直接放弃 `Δq_d` 这一路输入（会掉一点精度）。

B.2 NEXT 模型

```
import torch, torch.nn as nn

class NEXT(nn.Module):
    def __init__(self, n_j=6, h=25, hid=128, mlp=256, drop=0.1):
        super().__init__()
        self.lstm = nn.LSTM(3 * n_j, hid, num_layers=2,
                            batch_first=True, dropout=drop)
        self.head = nn.Sequential(
            nn.Linear(hid, mlp), nn.GELU(), nn.Dropout(drop),
            nn.Linear(mlp, mlp), nn.GELU(), nn.Dropout(drop),
            nn.Linear(mlp, n_j),
        )
        self.h = h

    def forward(self, x):            # x: (B, H, 3*n_j)
        out, _ = self.lstm(x)        # 隐状态每条序列重置 → 无状态滑窗
        return self.head(out[:, -1])  # 只预测当前步 tau_f
```

B.3 FIRST 采样器

```
import numpy as np

def phase_labels(tau_ext, qd, T_high, T_low, fps=50, pre_sec=1.0):
    """tau_ext: (N, 6) 外部力矩估计；qd: (N, 6) 角速度（用于剔除静止段误检）"""
    f = np.linalg.norm(tau_ext, axis=1)
    c = np.zeros(len(f), dtype=np.int8)
    on = False
    for i, v in enumerate(f):
        if not on and v >= T_high:
            on = True
        elif on and v <= T_low:
            on = False
        c[i] = int(on)
    s = np.zeros(len(f), dtype=np.int8)                # 0=FS 1=PC 2=C
    s[c == 1] = 2
    F = int(pre_sec * fps)
    onsets = np.where((c[1:] == 1) & (c[:-1] == 0))[0] + 1
```

```
    for t in onsets:
        s[max(0, t - F):t] = np.where(s[max(0, t - F):t] == 0, 1, s[max(0, t - F):t])
    return s

def make_weights(s, w=(1.0, 5.0, 1.0)):
    table = np.array(w)
    return table[s]
```

B.4 阈值标定

```
def sweep_threshold(f, labels, highs, lows):
    """在自由段/接触段已人工标注的数据上网格搜索 F1 最优的 (T_high, T_low)。"""
    best = (-1, None)
    for hi in highs:
        for lo in lows:
            if lo >= hi:
                continue
            pred, on = np.zeros_like(labels), False
            for i, v in enumerate(f):
                if not on and v >= hi:
                    on = True
                elif on and v <= lo:
                    on = False
                pred[i] = on
            tp = ((pred == 1) & (labels == 1)).sum()
            fp = ((pred == 1) & (labels == 0)).sum()
            fn = ((pred == 0) & (labels == 1)).sum()
            f1 = 2 * tp / max(2 * tp + fp + fn, 1)
            if f1 > best[0]:
                best = (f1, (hi, lo))
    return best
```

附录 C　交付文件与官方资料

文件	说明
PiPER_FACTR2_复现说明报告.docx	本报告（可编辑）
PiPER_FACTR2_复现说明报告.pdf	本报告 PDF
paper/piper_repro_report.md	本报告的 Markdown 源（数值与表格的唯一来源）
piper_docs/PiPER_标准版_官方参数与使用说明.md	官方参数与使用说明（整理版）
piper_docs/PiPER_Quick_Start_EN_v1.0.pdf	官方快速上手手册
piper_docs/PiPER_用户手册_中文_v1.1.0.pdf	官方用户手册（中文）
piper_docs/PiPER_CAN协议_API手册_zh_v1.5.xlsx	官方 CAN 协议（复现核心）
FACTR2_论文解析报告.docx / .pdf	论文本身的解析报告（背景与结果全貌）
paper/2606.12406v2.pdf	论文原文

官方资源	链接
产品页	https://global.agilex.ai/products/piper
Python SDK	https://github.com/agilexrobotics/piper_sdk
ROS 1 / ROS 2	https://github.com/agilexrobotics/Piper_ros
论文项目页	https://jasonjzliu.com/factr2
论文 arXiv	https://arxiv.org/abs/2606.12406

> 引用说明：本报告中所有论文数值均引自 arXiv:2606.12406v2 的正文表格与附录；硬件参数引自 AgileX 官方快速上手手册、用户手册与 CAN 协议手册；标有「建议 / 工程经验」的内容为复现补充，非论文或厂家原文。